

```
%matplotlib inline

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
accuracy_score
def Power():
    print(2**3)
Power()
8
```

(b) Create a new function, Power2(), that allows you to pass any two numbers, x and a, and prints out the value of x^a . You can do this by beginning your function with the line `def Power2(x, a):`. You should be able to call your function by entering, for instance, `Power2(3, 8)` on the commandline. This should output the value of 38, namely, 6, 561.

```
def Power2(x, a):
    print(x**a)
Power2(3, 8)
6561
```

(c) Using the Power2() function that you just wrote, compute 103, 817, and 1313.

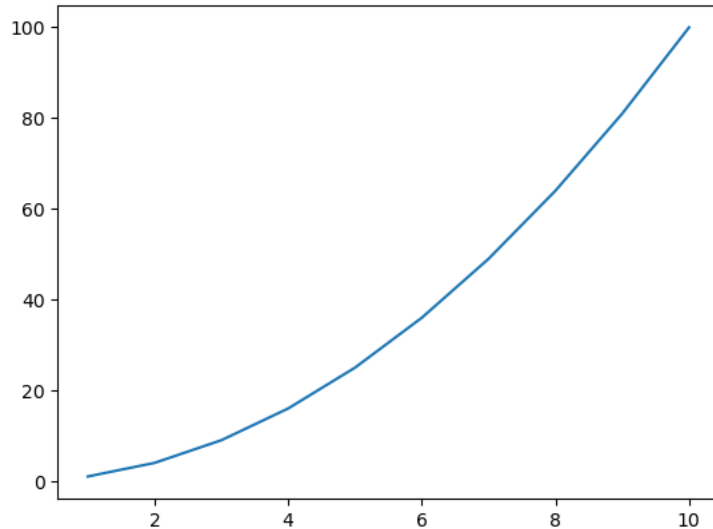
```
def Power2(x, a):
    print(x**a)
Power2(10, 3)
1000
def Power2(x, a):
    print(x**a)
Power2(8, 17)
2251799813685248
def Power2(x, a):
    print(x**a)
Power2(131, 3)
2248091
```

(d) Now create a new function, Power3(), that actually returns the result xa as a **Python object**, rather than simply printing it to the screen. That is, if you store the value xa in an object called result within your function, then you can simply return return this result, using the following line: `return result` Note that the line above should be the last line in your function, and it should be indented 4 spaces.

```
def Power3(x, a):
    if type(x) == list or type(x) == range:
        return np.power(integers, a)
    return x**a
```

(e) Now using the `Power3()` function, create a plot of $f(x) = x^2$. The x-axis should display a range of integers from 1 to 10, and the y-axis should display x^2 . Label the axes appropriately, and use an appropriate title for the figure. Consider displaying either the x-axis, the y-axis, or both on the log-scale. You can do this by using the `ax.set_xscale()` and `ax.set_yscale()` methods of `.set_xscale()` the axes you are plotting to. `.set_yscale()`

```
integers = range(1, 11, 1)
type(integers)
plt.plot(integers, Power3(integers, 2))
[<matplotlib.lines.Line2D at 0x232df4025d0>]
```



(f) Create a function, `PlotPower()`, that allows you to create a plot of x against x^a for a fixed a and a sequence of values of x . For instance, if you call `PlotPower(np.arange(1, 11), 3)` then a plot should be created with an x-axis taking on values 1, 2,...,10, and a y-axis taking on values 1, 8,...,1000.

```
def PlotPower(rangelist, power):
    powerlist = Power3(rangelist, power)
    plt.plot(rangelist, powerlist)
```

```
PlotPower(range(1, 11, 1), 3)
```

